Deprecate implicit conversions between `char8_t`, `char16_t`, and `char32_t`

Document number:
P3695R0

Date:
2025-05-18

Audience:
EWG, SG16

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Author:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3695R0/github

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/deprecate-unicode-conversion.cow

---

ABSTRACT: Implicit conversions between `char8_t`, `char16_t`, and `char32_t` are bug-prone and thus harmful to the language. I propose to deprecate them.

§ Contents

§ 1. Introduction

Implicit conversions between `char8_t` and `char32_t` invite bugs:

BUG: Until *very* recently, no major compiler would detect the following "bad comparison":

```
constexpr bool contains_oe(std::u8string_view str) {
    for (char8_t c : str)
        if (c == U'ö')
            return true;
    return false;
}
static_assert(contains_oe(u8"ö")); // fails?!
```

`c == U'ö'` always fails if `c` is a UTF-8 code unit because it is equivalent to `c == char32_t(0xf6)`, and a UTF-8 code unit cannot have this value.

BUG: An even more evil variation is a search which yields false positives:

```
constexpr bool contains_nbsp(std::u8string_view str) {
    for (char8_t c : str)
        if (c == U'\N{NO-BREAK SPACE}')
            return true;
    return false;
}
static_assert(contains_nbsp(u8"\N{CYRILLIC CAPITAL LETTER EL WITH MIDDLE
```

The assertion succeeds because Ӆ (U+0520) is UTF-8 encoded as `0xd4`, `0xa0`, and NBSP is U+00A0, so the `char32_t(0xa0)` value matches the second UTF-8 code unit of U+0520.

BUG: Such bad comparisons often don't occur directly, but within `<algorithm>`:

```
constexpr bool is_umlaut(char32_t c) {
    return c == U'ä' || c == U'ö' || c == U'ü';
}
// ...
```

```
constexpr std::u8string_view umlauts = u8"äöü";
static_assert(std::ranges::find_if(umlauts, is_umlaut) != umlauts.end());
```

Note that the "bad comparison" occurs between two `char32_t` in `is_umlaut`, which demonstrates that implicit conversions in general are bug-prone, not just comparisons. We obviously don't want to deprecate `char32_t == char32_t`.

Conversions "the other way" (e.g. `char32_t` → `char8_t`) are obviously bug-prone too because information is lost, but such bugs can already be caught by all major compilers' warnings, and they are problematic for the same reason as `int` → `short`, not because of anything specific to character types. The listed bugs are interesting *precisely because* no information is lost.

1.1. It's not hypothetical. This really happens.

These kinds of bugs are not far-fetched hypotheticals either; I have written such bugs myself, and have had them contributed to my syntax highlighter [µlight], which makes extensive use of `char8_t` and `char32_t`. Very early in development, I have realized how dangerous these implicit conversions are, so most functions in the style of `is_umlaut` have a deleted overload:

```
constexpr bool is_umlaut(char8_t) = delete;
constexpr bool is_umlaut(char32_t c) {
    return c == U'ä' || c == U'ö' || c == U'ü';
}
```

NOTE: Compilers do have warnings which detect comparisons which are always `false`, but technically, `char8_t` can have the values `0xf6` and `0xa0`, so it is undetectable.

1.2. The underlying problem

The underlying problem is that `char8_t == char32_t` is `Car == Banana`. In general, it is meaningless to compare code units with different encodings.

To be fair, Unicode character types aren't strictly required to store Unicode code units. However, that is their primary purpose, and the assumption holds true for any Unicode *character-literal* and *string-literal*.

2. Scope

I propose to deprecate implicit conversions between `char8_t`, `char16_t`, and `char32_t`. As demonstrated above, these are extremely bug-prone.

2.1. What about "safe" comparisons?

In comparisons between code units, certain ranges of code points yield the expected result. For example, `u8'x' == U'x'` is `true` because all Unicode encodings are ASCII-compatible, so the numeric value of anything in the basic latin block (≤ U+007F) will have the same single-code-unit value in UTF-8, UTF-16, and UTF-32.

However, even those should be deprecated because:

- Keeping these valid would essentially leak implementation details of Unicode encodings into the C++ core language, which seems like unclean design.
- To rely on this "feature", the developer needs to memorize which code points are "safe to use". It is not obvious whether `c == U'€'` or `c == U'$'` are always safe (hint: the latter one is), and it's quite likely that someone uses this "feature" accidentally.
- It would make this "feature" (or lack thereof) harder to teach than it needs to be. The rule can be very simple: different `charN_t` cannot be converted to one another. Simple rules are easy to teach.

§ 2.2. What about `char` and `wchar_t`?

`char` and `wchar_t` have existed for too long to make any deprecation of their behavior realistic at this point. There are approximately ten trillion lines of C++ code using `char`[*citation needed*].

It would still be plausible to deprecate say, conversions between `char` and `charN_t`. However, there's a good chance that these are valid because UTF-8 text is often stored in `char[]`, and UTF-16 or UTF-32 text is often stored in `wchar_t[]`. On the contrary, `char8_t` and `char32_t` almost certainly use different encodings.

§ 2.3. What about conversions with integers?

It is quite common to compare character types to integer types. For example, we may write `c <= 0x7f` to check whether a character falls into the basic latin block. There is nothing exceptionally bug-prone about comparing with say, `0x00A0` instead of `U'\u00A0'`, so we are not interested in deprecating character/integer conversions.

§ 2.4. What comes after deprecation?

The goal is to eventually remove these conversions entirely. Since the behavior is easily detected (§4. Implementation experience) and easily replaced (§3.1. Replacement for deprecated behavior), removal should be feasible within one or two revisions of the language.

Furthermore, I don't believe that having "tombstone behavior" would be necessary. That is, allowing the conversion to happen but making the program ill-formed if it happens. The reason is that `char8_t`, `char16_t`, and `char32_t` rarely appear in overload sets that include types that are not characters.

> EXAMPLE: Without "tombstone behavior", the following code would eventually change its meaning:

```cpp
void f(std::any);
void f(char32_t);

int main() {
    // Currently selects f(char32_t), would select f(std::any) in the fu
    f(u8'a');
}
```

§ 3. Impact on existing code

It is not trivial to estimate how much code would be affected by a deprecation like this. However, that is ultimately not what makes or breaks this proposal. The goal is not to deprecate a rarely used feature to give it new meaning, like `array[0,1]` prior to [P1161R3].

The goal is to deprecate a bug-prone and harmful feature to make the language safer.

The longer we wait, the more mistakes will be made using `char8_t` and other types. C++ will undoubtedly get improved support for the Unicode character types over time, making them used more frequently, so we better deal with this problem now than never.

§ 3.1. Replacement for deprecated behavior

If the new deprecation warnings spot a bug like in §1. Introduction, some work will be required to fix it, but the deprecation will have done its job.

If the comparison is obviously safe, such as `c == U'0'` with `char8_t c`, the resolution is usually trivial, like `c == u8'0'`. This could even be done automatically with tools like clang-tidy.

§ 4. Implementation experience

Corentin Jabot has recently implemented a `-Wcharacter-conversion` warning in Clang ([ClangWarning]), which is enabled by default. You can test this at [CompilerExplorer].

However the warning is more conservative than the proposed deprecation; it does not warn on "safe comparisons" (§2.1. What about "safe" comparisons?).

§ 5. Wording

The following changes are relative to [N5008].

Change [basic.fundamental] paragraph 9 as follows:

> The types `char8_t`, `char16_t`, and `char32_t` are collectively called *Unicode character types.* Type `char8_t` denotes a distinct type whose underlying type is `unsigned char`. Types `char16_t` and `char32_t` denote distinct types whose underlying types are `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`.

Change [conv.integral] paragraph 1 as follows:

> A prvalue of an integer type `S` can be converted to a prvalue of another integer type `D`. The conversion is deprecated ([depr.conv.unicode]) if
>
> - `S` and `D` are two different Unicode character types ([basic.fundamental]) and
> - the conversion is not necessitated by a `static_cast` ([expr.static.cast]).
>
> [*Note*: This deprecation also applies to cv-qualified Unicode character types because prvalues of such types are adjusted to cv-unqualified types; see [expr.type]. — *end note*]
> ~~A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.~~

Insert a new paragraph immediately following [conv.integral] paragraph 1:

> A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.

Change [expr.arith.conv] paragraph 1 as follows:

> Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
>
> - The lvalue-to-rvalue conversion ([conv.lval]) is applied to each operand and the resulting prvalues are used in place of the original operands for the remainder of this section.
> - [...]
> - Otherwise, each operand is converted to a common type `C`. The conversion is deprecated if the operands are of two different Unicode character types ([depr.conv.unicode]). The integral promotion rules ([conv.prom]) are used to determine a type `T1` and type `T2` for each operand. Then the following rules are applied to determine `C`:
>
>   - [...]

Insert a new subclause in [depr] between [depr.local] and [depr.capture.this], containing a single paragraph:

Unicode character conversions [depr.conv.unicode]

The following conversions are deprecated:

- Integral conversions ([conv.integral]) not necessitated by a `static_cast`, where the source type and destination type are two different Unicode character types ([basic.fundamental]).
- Usual arithmetic conversions ([expr.arith.conv]) where the operands after lvalue-to-rvalue conversion ([conv.lval]) are two different Unicode character types.

[*Example*:

```
bool is_oe(char8_t c) {
    return c == U'ö';                       // deprecated
}
void f() {
    char32_t c = u8'x';                     // deprecated
    char32_t c = 'x';                       // OK, char is not a Unicode character
    is_oe(U'ö');                            // deprecated
    is_oe(static_cast<char8_t>(U'ö'));      // OK, integral conversion necessitated
    is_oe((char8_t)U'ö');                   // OK, explicit conversion performed by
}
```

— *end example*]

§ 6. References

[N5008] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-03-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf

[µlight] *Jan Schultke.* ascii_chars.hpp utilities in µlight
https://github.com/Eisenwave/ulight

[ClangWarning] *Corentin Jabot.* [Clang] Add warnings when mixing different charN_t types
https://github.com/llvm/llvm-project/pull/138708

[CompilerExplorer] Demonstration of -Wcharacter-conversion
https://compiler-explorer.com/z/8j9qqe8MY

[P1161R3] *Corentin Jabot.* Deprecate uses of the comma operator in subscripting expressions
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1161r3.html